

Tipos de Dados, Variáveis e Constantes

Python como calculadora

- O interpretador Python (e.g. IDLE) pode ser usado como uma calculadora.
- Por exemplo, as quatro operações aritméticas são denotadas pelos símbolos:

+ adição

- subtração

* multiplicação

/ divisão

- Ainda temos as operações:

// divisão inteira

% resto da divisão

Python como calculadora

```
>>> 10
10

>>> # Um comentário é precedido do caractere "#".
>>> 10 + 5
15

>>> 10 - 15 # Comentário pode aparecer também na linha do código.
-5

>>> 10 * 3
30

>>> 10 / 3      # Divisão com barra única retorna um float.
3.3333333333333335

>>> 10 // -3     # Divisão inteira (barra dupla) retorna o piso.
-4

>>> 10 % 3      # Resto de divisão inteira simbolizado por %.
1
```

Tipos de dados

- **Tipo de dado** é uma forma de categorizar valores para serem processados de forma semelhante.
 - Os números inteiros são processados de forma diferente dos números decimais (ponto flutuante) e dos complexos.
- **Tipo primitivo** é aquele já embutido no núcleo da linguagem.
 - **Simples**: numéricos (*int*, *long*, *float*, *double*, *complex*), cadeias de caracteres (*str*) e lógicos (*bool*);
 - **Compostos**: listas, tuplas, dicionários e conjuntos.
- **Tipos definidos pelo usuário** são correspondentes a classes (orientação objeto).

Variáveis: conceitos

- **Variáveis** são áreas de memória reservadas para armazenar dados.
- O nome que damos a uma variável é o seu identificador:
 - Os nomes podem compor letras (a-z ou A-Z), número (0-9) e/ou sublinhado (`_`);
 - O primeiro caractere não pode ser um número;
 - Não podem ser palavras reservadas (*if*, *while*, *for*, *int*, etc.)
- Variáveis servem para:
 - Guardar valores intermediários;
 - Construir estruturas de dados.

Variáveis: declaração

- Uma variável é modificada usando o comando de atribuição:
 - Forma geral:

var = expressão

- Onde:
 - ***var*** é o identificador (nome) da variável.
 - ***expressão*** é o valor atribuído a variável.
- É possível também fazer múltiplas atribuições:

var1, var2, ..., varN = expr1, expr2, ..., exprN

Variáveis: exemplos

```
>>> a = 1
```

```
>>> a
```

```
1
```

```
>>> a = 2 * a
```

```
>>> a
```

```
2
```

```
>>> a, b = 3, 2
```

```
>>> a, b
```

```
(3, 2)
```

```
>>> a, b = 3*a, a
```

```
>>> a, b
```

```
(9, 3)
```

```
>>> a, b = b, a
```

```
>>> a, b
```

```
(3, 9)
```

```
>>> a, b = b, a
```

```
>>> a, b
```

```
(9, 3)
```

Variáveis: observações

- Variáveis são criadas dinamicamente e destruídas quando não são mais necessárias, e.g., quando saem de escopo.
- O tipo de uma variável muda conforme o valor atribuído, i.e., **int**, **float**, **string**, **boolean** e etc.
 - Não confundir com linguagens sem tipo.

```
>>> a = "1"
```

```
>>> b = 1
```

```
>>> a + b
```

```
Traceback (most recent call last):
```

```
File "<pyshell#7>", line 1, in <module> a + b
```

```
TypeError: can only concatenate str (not "int") to str
```


Números inteiros

- Constantes podem ser informadas em diferentes bases, alterando a notação utilizada para escrevê-las.
 - Hexadecimal: preceder dígitos com 0x
 - Octal: preceder dígitos com 0o
 - Binário: preceder dígitos com 0b

>>> 0x41	>>> 0o22
65	18
>>> 0x10	>>> 0b1111
16	15
>>> 0x1f	>>> 0b10
31	2

Números de ponto flutuante

- São codificados como o tipo *double* – tipicamente usam 2 palavras (64 bits).
- Constantes *double* têm que possuir um ponto decimal ou serem escritas em notação científica com a letra “e” (ou “E”) precedendo a potência de base 10.

>>> 10	# inteiro
10	
>>> 10.0	# ponto flutuante
10.0	
>>> 99e3	# mesmo que 99*10**3
99000.0	
>>> 99e-3	# mesmo que 99*10**(-3)
0.099	

Números complexos

- Representados com dois números de ponto flutuante: um para a parte real e outro para a parte imaginária.
- Constantes são escritas como uma soma, sendo que a parte imaginária tem o sufixo `j` ou `J`.

```
>>> 1+2j
      (1+2j)
>>> 1+2j*3
      (1+6j)
>>> (1+2j)*3
      (3+6j)
>>> (1+2j)*3j    # j ao quadrado é igual a -1
      (-6+3j)
```

Expressões booleanas

- Ou **expressões lógicas**, resultam em **True** (verdadeiro) ou **False** (falso).
 - As constantes **True** e **False** são apenas símbolos convenientes.
- São muito usadas em comandos condicionais e de repetição.
- Servem para analisar o estado de uma computação e permitir escolher um próximo passo.
- Operadores mais usados:
 - Relacionais: `>` , `<` , `>=` , `<=` , `==` e `!=`
 - Booleanos: **and**, **or** e **not**

Expressões booleanas

- Em Python, a avaliação das expressões lógicas é feita em formato de “Curto-circuito”
 - Expressão avaliada da esquerda para a direita.
 - Se o resultado (**True** ou **False**) puder ser determinado sem avaliar o restante da expressão, esse resultado é retornado imediatamente.
- Os valores **None**, **0** (zero) ou **False** é visto como falso (**False**) e qualquer outro valor é visto como verdadeiro (**True**).

Expressões booleanas

>>> 1 == 1 True	>>> not 1 < 2 False
>>> 1 == 2 False	>>> not 1 < 2 or 2 < 3 True
>>> 1 == 1 or 1 == 2 True	>>> not (1 < 2 or 2 < 3) False
>>> 1 == 1 and 1 == 2 False	>>> "alo" and 1 1
>>> 1 < 2 and 2 < 3 True	>>> "alo" or 1 'alo'

Expressões booleanas

- Os operadores **or** e **and** não retornam necessariamente **True** e **False**, retorna o valor do operando.
 - O operador **or** retorna o primeiro operando se for vista como **True**, caso contrário retorna o segundo.
 - O operador **and** retorna o primeiro operando se for vista como **False**, caso contrário retorna o segundo.
- Operadores relacionais são avaliados antes de **not**, que é avaliado antes de **and**, que é avaliado antes de **or**.

Expressões booleanas

```
>>> 0 or 100
100
```

```
>>> False or 100
100
```

```
>>> "abc" or 1
'abc'
```

```
>>> 1 and 2
2
```

```
>>> 0 and 3
0
```

```
>>> False and 3
False
```

```
>>> 1 and 2 or 3
2
```

```
>>> 0 or 2 and 3
3
```

```
>>> 1 and not 0
True
```

```
>>> 0 or not 1
0
```

Strings

- **String** é uma cadeia de caracteres.
- Constantes string são escritas usando aspas simples ou duplas.
 - "PYTHON" ou 'PYTHON'
- O operador "+" é usado para concatenar strings.
 - "P" + "Y" é o mesmo que "PY"
- O operador "*" pode ser usado para replicar strings.
 - "P" * 10 é o mesmo que "PPPPPPPPPP" ("P" 10 vezes).

Strings

- Python usa a codificação UTF-8 por padrão.
 - Padrão mundial que codifica caracteres na internet. Representa mais de um milhão de símbolos (acentos, emojis e caracteres de vários idiomas) usando de um a quatro bytes.
- Caracteres não imprimíveis podem ser expressos pela notação “barra-invertida” (\). Na função `print()`:
 - `\n` é o mesmo que *new line*
 - `\r` é o mesmo que *carriage return*
 - `\t` é o mesmo que *tab*
 - `\b` é o mesmo que *backspace*
 - `\\` é o mesmo que `\`
 - `\x41` é o caractere de código hexadecimal 41 ('A')
 - `\x61` é o caractere de código hexadecimal 61 ('a')

Strings

<pre>>>> "ab\r d" 'ab\r d' # chars não imprimíveis >>> print("ab\r d") ab d >>> print("abc\td") abc d >>> print("abc\nd") abc d</pre>	<pre>>>> print("abc\\nd") abc\nd # \b deve ser o backspace >>> print("ab\bc") ac # i não é a letra i >>> print("\x41\xA1") Ai</pre>
---	--

Strings

- A notação barra-invertida (\) pode ser desabilitada desde que a constante string seja precedida por um r (erre minúsculo)
 - São chamadas *strings raw* (cruas). Exemplo:

```
>>> print("abc\ncd\tef")
abc
cd      ef

>>> print(r"abc\ncd\tef")
abc\ncd\tef
```

Strings

- Constantes string podem ser escritas com várias linhas desde que as aspas não sejam fechadas e que cada linha termine com uma barra invertida ("\\"). Exemplos:

```
# Exemplo 1
>>> print("abcd\n\
... efgh\n\
... ijk")
abcd
efgh
ijk
```

```
# Exemplo 2
>>> print("abcd\
... efgh\
... ijk")
abcdefghijkl
```

Strings

- Também é possível escrever constantes string em várias linhas incluindo as quebras de linha usando três aspas duplas (ou simples) como delimitadores. Exemplos:

```
# Com aspas duplas
>>> print("""
... Um tigre
... dois tigres
... três tigres""")
Um tigre
dois tigres
três tigres
```

```
# Com aspas simples
>>> print('''abcd
... efgh''')
abcd
efgh
```

Strings: índices

- Endereçam ou indexam caracteres individuais de uma string.
 - Notação: `nome_da_string[índice]`
 - O primeiro caractere tem índice `0`
 - O último caractere tem índice `-1`

```
>>> ies = "UESPI"
```

```
>>> ies[0]
'U'
```

```
>>> ies[2]
'S'
```

```
>>> ies[-1]
'I'
```

0	1	2	3	4
U	E	S	P	I

Strings: fatias (slices)

- **Slice:** notação usada para separar trechos de uma string.
 - Formato: `nome_string[índice_1 : índice_2 : salto]`
 - Retorna os caracteres a partir do `índice_1` (inclusive) até o `índice_2` (exclusive).
 - O parâmetro `salto` indica a quantidade de caracteres a serem saltados entre `índice_1` e `índice_2`.
 - O salto padrão é 1.
 - Se `índice_1` é omitido, é assumido 0 (zero).
 - Se `índice_2` é omitido, é assumido o fim da string.
 - Lembrando: índice inicial é 0 (zero) e índice final é -1

Strings: fatias (slices)

<pre># Exemplos >>> ies = "UESPI" >>> ies 'UESPI' >>> ies[0:2] 'UE' >>> ies[2:] 'SPI'</pre>	<pre>>>> ies[:] 'UESPI' >>> ies[:-1] 'UESP' >>> ies[-1:] 'I'</pre>
--	---

Strings: fatias (slices)

<pre># Exemplos >>> ies = "UESPI Parnaíba" >>> ies 'UESPI Parnaíba' >>> ies[0:10:1] 'UESPI Parn' >>> ies[0:10:2] 'USIPr'</pre>	<pre>>>> ies[::] 'UESPI Parnaíba' >>> ies[::-1] 'abíanraP IPSEU' >>> ies[::2] 'aína PE'</pre>
---	--

Funções embutidas

- Além dos operadores, existem funções para computar dados.
- Funções podem ser definidas:
 - Pelo programador.
 - Em módulos da biblioteca padrão.
 - Por *default*: são as funções embutidas (*built-in*).
- Na verdade, fazem parte do módulo **__builtins__**, que é sempre importado em toda aplicação. Exemplos:
 - `abs(n)` retorna o valor absoluto do número `n`.
 - `chr(n)` retorna uma string com um único caractere cujo código ASCII é `n`.
 - `ord(ch)` retorna o código ASCII do caractere `ch`.
 - `pow(n1, n2)` retorna o valor de `n1` elevado a `n2` (potência).

Funções embutidas

```
>>> abs(10)
10
```

```
>>> abs(-19)
19
```

```
>>> chr(95)
'_'
```

```
>>> chr(99)
'c'
```

```
>>> ord('A')
65
```

```
>>> ord('a')
97
```

```
>>> pow(3, 2)
9
```

```
>>> pow(2, 3)
8
```

Importando módulos

- Muitas funções importantes são disponibilizadas em módulos da biblioteca padrão.
 - Ex.: o módulo **math** tem funções como **sin**, **cos**, **exp** ...
- Um módulo pode conter não só funções mas também variáveis ou classes.
 - Ex.: o módulo **math** define a constante **pi**
- Para usar os elementos de um módulo, pode-se usar o comando **import**, nos seguintes formatos:

```
import modulo
from modulo import nome1, ..., nomeN
from modulo import *
```

Importando módulos: exemplos

```
import math
# importa todos os elementos do módulo math, mas todos
# os elementos devem ser citados precedidos por math.
```

```
from math import *
# importa todos os elementos do módulo math
```

```
from math import sin
# importa apenas a função sin do módulo math
```

Importando módulos: exemplos

```
>>> import math
>>> a = sin(30)
Traceback (most recent call
last):
File "<stdin>", line 1, in ?
NameError: name 'sin' is not
defined
```

```
>>> a = math.sin(30)
>>> from math import sin
>>> a = sin(30)
>>> print(a)
-0.9880316240928618
```

```
>>> a = sin(radians(30))
Traceback (most recent call
last):
File "<stdin>", line 1, in ?
NameError: name 'radians' is
not defined
```

```
>>> from math import *
>>> a = sin(radians(30))
>>> a
0.49999999999999994
```

Explorando módulos

- Para saber mais sobre alguma função, peça ajuda.

```
>>> import math
>>> help(math.cos)
```

Help on built-in function cos in module math:

```
cos(x, /)
Return the cosine of x (measured in radians).
```

Explorando módulos

- **Desafio:** Elabore um código que sorteia aleatoriamente um número e o armazena em uma variável. Em seguida, solicite ao usuário para tentar adivinhar o número sorteado. O programa deve pedir ao usuário para informar um número e em seguida dizer se o usuário acertou ou não.